

# Expectation-Guided Self-Verification for Aligning Large Reasoning Models with Domain Knowledge

Han Zhang\*  
hanzhang24@m.fudan.edu.cn  
Fudan University  
Shanghai, China

Chen Zhao\*  
czhao25@m.fudan.edu.cn  
Fudan University  
Shanghai, China

Shasha Wang  
wangshasha@pjlab.org.cn  
Shanghai Artificial Intelligence  
Laboratory  
Shanghai, China

Gang Chen<sup>†</sup>  
chengang050970@foxmail.com  
Fudan University  
Shanghai, China

Chenghong Zhang  
chzhang@fudan.edu.cn  
Fudan University  
Shanghai, China

## Abstract

Large Reasoning Models (LRMs) have demonstrated impressive performance across scientific and coding tasks, yet they often struggle with mismatches between their reasoning and users' domain-specific knowledge, particularly in specialized fields like medical consultation services. We study domain-specific reasoning, defined as producing responses whose prioritized evidence and actions match expert expectations for a query in a target domain. Existing methods primarily focus on improving the LRM's chain-of-thought or retrieving external relevant information to augment its contextualized responses. However, these methods largely fail to explicitly identify and address users' underlying domain-specific needs. To bridge this gap, we propose a novel Expectation-Guided Self-Verification (XGSV) framework, inspired by the Expectation Confirmation Theory, which implies that user satisfaction with the LRM's responses depends on the alignment between their pre-formed expectations and the actual responses provided. Specifically, XGSV first generates an expectation-guided knowledge graph based on user queries by activating a pretrained query-to-graph encoder. Simultaneously, the LRM's chain of-thought is transformed into a reasoning knowledge graph using the same architecture. These two graphs are then aligned and refined through an adaptive graph fusion module for self-verification. Finally, the verified knowledge graph is encoded into a learnable soft prompt, which guides the LRM to generate improved responses that better address users' implicit domain-specific needs. Experimental results based on real-world datasets show that XGSV outperformed existing baseline methods, demonstrating the effectiveness of the expectation-guided self-verification in enhancing the contextualized reasoning capabilities of LRMs. Our code and dataset are available in an online repository: <https://github.com/chzhao01/XGSV>.

\*Both authors contributed equally to this research.

<sup>†</sup>Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*  
© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792638>

## CCS Concepts

• Computing methodologies → Knowledge representation and reasoning.

## Keywords

Large Reasoning Models, Large Language Model Application, Knowledge Graphs, Domain Reasoning, Reasoning Process Alignment

## ACM Reference Format:

Han Zhang, Chen Zhao, Shasha Wang, Gang Chen, and Chenghong Zhang. 2026. Expectation-Guided Self-Verification for Aligning Large Reasoning Models with Domain Knowledge. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792638>

## 1 Introduction

Large Reasoning Models (LRMs) have witnessed rapid advancements and widespread applications across various domains, including code generation and scientific problem solving [11, 13, 23]. Notably, LRMs possess the ability to generate a thoughtful chain of reasoning before arriving at a final answer. This advanced reasoning capability positions LRMs as pivotal tools in boosting the efficiency and adaptability of AI applications in real-world scenarios [7, 28].

However, in knowledge-intensive downstream application domains, such as medical consultation and financial risk management, LRMs often struggle to align their reasoning processes with in-depth, domain-specific user expectation, resulting in the misalignment between the reasoning logic of LRMs and the cognitive logic of users. This misalignment is a fundamental cause of suboptimal LRM responses and is typically manifested as a gap between the user's expected response and the LRM's actual output. Figure 1 presents the reasoning misalignment issue in mainstream LRMs. As shown on the left, we prompt DeepSeek-R1 [8] to respond to a medical query. The model generates a specific chain of thought that includes a comprehensive list of laboratory tests, but fails to align its reasoning process with the user's domain-specific expected (required) responses—identifying the most critical examination to perform. Such reasoning, along with its output, is overly broad, lacks prioritization, and does not adequately align with the user's clinical need for rapid decision-making. In those high-stakes contexts, e.g.,

neonatal jaundice diagnosis, such reasoning misalignment may even compromise patient safety.

Several existing methodologies hold the potential to enhance the domain-specific reasoning capabilities of LRMs. Test-time scaling methods, such as self-refine [18] and self-consistency [29], enhance LRMs' responses by generating multiple reasoning paths and optimizing outputs through consensus selection or iterative refinement during inference. Finetuning methods, including SFT methods like LoRA [10], Prefix-Tuning [15], inject reasoning functions or domain-specific reasoning patterns into the model. Besides, RL fine-tuning methods, e.g., GRPO [23], refine models' reasoning patterns guided by reward signals. Knowledge Graph-based methods, like Think on Graph [24], incorporate structured domain knowledge to align models' reasoning steps with knowledge graphs, thereby improving their factual grounding and domain knowledge utilization.

Despite some progress made by existing studies, their developed methods exhibit significant limitations in addressing the misalignment between user expectation and model reasoning. As shown in Figure 1, test-time scaling methods only optimize for internal consistency within the model-generated reasoning paths but do not explicitly account for domain-specific user expectations, especially in knowledge-intensive and high-stakes domains. SFT methods tend to overfit specific reasoning patterns and lack generalization. Meanwhile, RL-based fine-tuning approaches are highly sensitive to reward models, limiting their applicability in domain knowledge-intensive scenarios. KG-based methods fail to dynamically adapt to users' context-aware evolving needs, as they rely on static knowledge graphs with pre-defined entities and rigid relationships that cannot capture dynamic user expectations. In summary, these approaches lack a unified mechanism to effectively align LRMs with the domain-specific and evolving user expectations.

To address these challenges, we propose a novel Expectation Guided Self-Verification (XGSV) framework that enables LRMs to self-verify and refine the generated reasoning chains. The framework design is inspired by the Expectation Confirmation Theory, which posits that individuals pre-form expectations before new experiences. Alignment between expectation and actual experience yields a positive cognition fitness and user satisfaction, while mismatch leads to a disconfirmation and dissatisfaction [20]. Specifically, we build a dynamic and expectation-guided knowledge graph aligning with the theory's construct "pre-formed expectations", which extracts structured user expectations from their queries using a pretrained query-to-graph encoder. In parallel, drawing from the theory's construct "actual experience", we convert the LRM's chain-of-thought into a structured dynamic reasoning knowledge graph using the same encoder architecture. To compare "expectation" with "actual experience" and generate "confirmation" signals, we further design an adaptive graph fusion module to align the experience graph with the expectation graph to realize expectation-guided self-verification of reasonings within the LRM. Finally, the verified knowledge graph is encoded into a learnable soft vector space that steers the LRM toward generating refined and self-verified responses that better satisfy users' implicit domain-specific expectations. The contributions of this study are as follows:

- **Theoretically**, inspired by Expectation-Confirmation Theory, we introduce a novel mechanism that refines LRM reasoning by aligning it with domain-specific user expectations in knowledge-intensive applications, bridging the gap between user expectation and model reasoning.
- **Methodologically**, we propose XGSV, an expectation-guided self-verification framework that constructs and aligns expectation graphs and reasoning graphs to verify and refine model reasoning.
- **Empirically**, extensive experiments demonstrate the superiority of XGSV over strong baselines, while ablation studies confirm the contribution of each component.

## 2 Related work

### 2.1 Large Reasoning Models

Research in AI has been placing greater emphasis on improving reasoning abilities in next-generation models [19], like Deepseek [8], OpenAI GPT [1], Qwen [34], LLaMa [6], Gemini [3], Kimi [26]. However, current LRMs still suffer from various issues, such as high hallucination rates [14, 25] and insufficient performance in downstream domains [27].

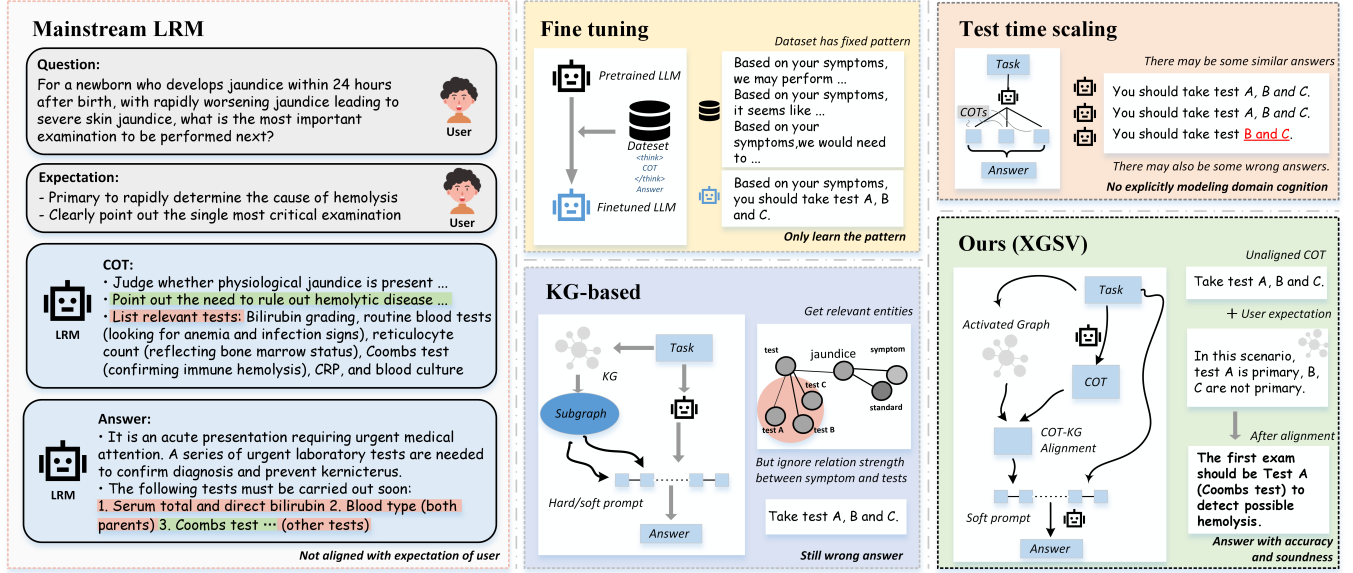
### 2.2 Improvement in LRMs

A wide range of methods have been proposed to enhance the reasoning abilities of LRMs across different settings. These efforts can be broadly categorized into three main paradigms.

**Test-time scaling methods.** Prompt-based approaches aim to improve reasoning performance during inference without modifying model parameters. Self-Consistency [29] samples multiple reasoning paths and selects the most consistent answer via majority voting, while Self-Refine [18] iteratively improves outputs by prompting the model to revise its own reasoning chain. Tree of Thoughts [35] further generalizes chain-of-thought reasoning by exploring multiple reasoning trajectories in a tree-structured search space. While effective in practice, these methods primarily rely on internal model heuristics and do not explicitly incorporate external or user-specific domain knowledge.

**Fine-tuning methods.** Fine-tuning approaches inject reasoning abilities into models through task supervision or feedback signals. Representative methods include LoRA [10], prefix-tuning [15], prompt tuning [12], and instruction tuning [30]. More recently, reinforcement learning methods such as GRPO [23], PPO [22], and DPO [21] have shown effectiveness in optimizing reasoning behaviors using reward-based feedback [8]. However, these approaches often suffer from training instability, limited interpretability, and reduced adaptability in knowledge-intensive domains.

**KG-based methods.** Knowledge graph (KG)-based methods integrate external structured knowledge to enhance factual consistency and relational reasoning. Thoughts over Graphs (ToG) [24] aligns reasoning steps with subgraphs extracted from a knowledge graph, grounding predictions in explicit relational structures. G-Retriever [9] retrieves relevant subgraphs using a PCST algorithm and incorporates them into LLMs via graph-encoded soft prompts, improving scalability and reducing hallucinations. Despite their interpretability advantages, these methods typically treat knowledge



**Figure 1: An illustrative case showing how LRM and existing methods fail to meet user expectations in neonatal jaundice. In contrast, XGSV aligns reasoning with user expectation, explicitly identifying the Coombs test as the primary examination.**

graphs as static resources and may struggle to adapt to dynamic or context-dependent user requirements.

### 2.3 Research Gap

Existing approaches for improving large reasoning models can be broadly summarized into test-time scaling, fine-tuning, and KG-based paradigms. Test-time scaling methods offer a practical way to boost inference performance without additional training, but largely rely on ad hoc heuristics and lack principled mechanisms for interpretability and robust generalization across domains. Fine-tuning methods internalize knowledge by updating model parameters, enabling task adaptation but often at the cost of stability, interpretability, and transferability. KG-based methods preserve model parameters and improve interpretability through structured prompting, yet their effectiveness is constrained by retrieval quality and limited robustness to evolving user expectations and out-of-distribution scenarios. In summary, current approaches lack effective and interpretable mechanisms for aligning LRMs with domain-specific and evolving user expectations.

In this work, we aim to bridge these gaps by introducing a new paradigm that directly integrates external knowledge into the reasoning process itself, enabling interpretable, adaptable, and expectation-aware reasoning during inference.

## 3 Methodology

Overall, Figure 2 illustrates the framework of our Expectation-Guided Self-Verification (XGSV) method, comprising two key parts: (1) Expectation-Guided knowledge Graph Generation and (2) Reasoning Knowledge Graph Verification. In the first part, XGSV employs an expectation-guided knowledge graph encoder composed of three steps: top-K entities retrieval, query-conditioned entity

representation, and expectation-guided graph activation. The encoder is trained via graph prompt tuning task, and is subsequently applied to predict expectation-guided knowledge graphs for user queries in the second stage. In the second part, XGSV first samples chain-of-thought from LRM and converts it into a reasoning knowledge graph with the same encoder architecture. Then an adaptive graph fusion mechanism verifies the reasoning knowledge graph against the expectation-guided knowledge graph. Finally, a graph soft prompt guides the large reasoning model’s generation process.

### 3.1 Problem Statement

**Motivation.** Let  $Q$  denote a user query. When prompted with  $Q$ , an LRM (e.g., DeepSeek-R1) generates a chain-of-thought COT to produce the final response  $R$ . In domain-specific reasoning tasks, however, such reasoning processes often fail to align with user expectations, leading to logical inconsistencies or omissions of domain-critical facts, and ultimately undermining reliability and interpretability.

**Assumptions.** We assume access to two complementary sources of domain knowledge. (1) Query-cot-answer triples, denoted as  $\mathcal{D}$ , reflect expert reasoning trajectories and provide supervision signals for aligning LRM reasoning with domain-specific practices, where  $\mathcal{D} = \{(Q_i, COT_i, A_i)\}_{i=1}^n$ . (2) Domain knowledge graphs  $\mathcal{G} = \{(h, r, t) \mid h, t \in \mathcal{E}, r \in \mathcal{R}\}$  encode structured factual knowledge with  $N$  entities and  $M$  relations, serving as an external backbone for factual grounding and relational consistency.

**Objective.** Our objective is to jointly leverage  $\mathcal{D}$  and  $\mathcal{G}$  to enhance the domain reasoning capability of LRMs. Specifically, we aim to integrate expert reasoning demonstrations from  $\mathcal{D}$  and structured knowledge from  $\mathcal{G}$ , enabling the model to generate reasoning chains that are *aligned with expert expectations, grounded in domain knowledge, and robust across diverse queries*.

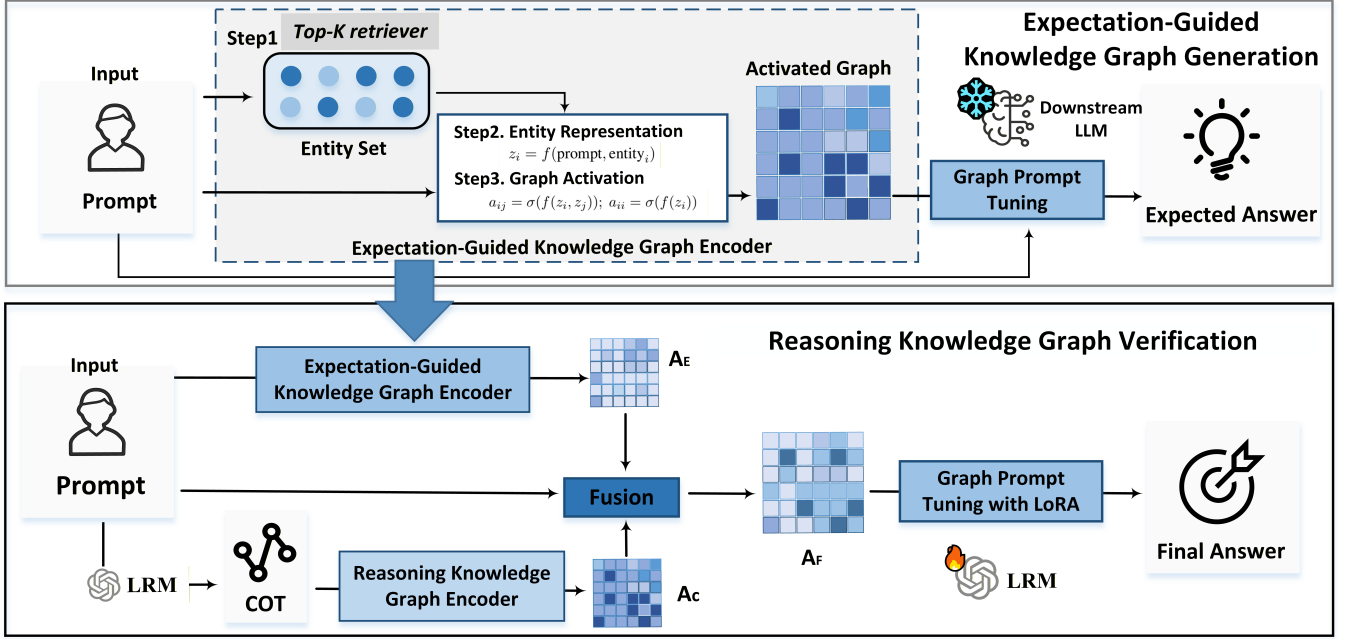


Figure 2: Overview of our proposed XGSV framework. Top: Expectation-Guided Graph Generation — Our module encodes user queries into structured graphs using Top-K entity retrieval, entity representation learning, and graph activation, with the entire encoding pipeline being optimized via graph prompt tuning. Bottom: Expectation-Guided Reasoning Verification - LRM’s reasoning chain is verified via adaptive graph fusion to refine its final answer.

### 3.2 Expectation-Guided Graph Generation

Expectation-Guided Graph Generation aims to instantiate, from a free-form user query  $Q$ , a compact subgraph that reflects what a domain expert would expect the model to recall and connect. Concretely, we embed  $Q$  and all entities, retrieve the Top-K candidates relevant to  $Q$ , and produce *query-conditioned* entity vectors that attend to pertinent parts of  $Q$ . Based on these vectors, we construct an *activation matrix*  $A$  whose diagonal entries quantify per-entity knowledge strength and whose off-diagonals capture pairwise relational strength. The resulting expectation-guided subgraph  $(\{z_i\}_{i=1}^K, A)$  is encoded by a lightweight GNN to yield a soft graph prompt compatible with the downstream LLM. Intuitively, this stage converts an unstructured query into a structured, expectation-aligned scaffold that is faithful to domain knowledge.

**3.2.1 User Query and Entity Embedding.** Our framework begins by embedding each user query  $Q$  and each knowledge entity  $V$  into continuous vector spaces. For the query embedding, we employ a pretrained LLM and extract its final hidden states to obtain  $Q \in \mathbb{R}^{d \times L}$ , where  $d$  is the hidden size and  $L$  is the token length. For each entity in  $\mathcal{V}$ , we generate a textual description using LLM APIs and encode the entity description into a last-token embedding using the same pretrained LLM. All entity embeddings are stacked column-wise as  $V = [v_1, v_2, \dots, v_N] \in \mathbb{R}^{d \times N}$ , where  $v_i \in \mathbb{R}^d$ .

**3.2.2 Expectation-Guided Knowledge Graph Encoder.** In this section, we introduce three steps towards accurate expectation-guided graph generation.

- **Step 1: Top-K Entities Retrieval**

Since user queries cannot cover all entities, we retrieve the Top-K most similar entities for a given problem  $Q$  to support subsequent algorithm modeling. Specifically, We use the last-token embedding  $q = Q_{[-1]} \in \mathbb{R}^d$  as the sentence embedding and compute cosine similarities with all entities  $\{v_n\}_{n=1}^N$ . Let  $I_K = \text{TopK}_K(\{\text{sim}(q, v_n)\}_{n=1}^N)$  be the index set of the Top-K most relevant entities. The retrieved entity set is  $\mathcal{V}^* = \{v_{(i)} \mid i \in I_K\}$  and the matrix  $V^* = [v_{(i)}]_{i \in I_K} \in \mathbb{R}^{d \times K}$ .

- **Step 2: Query-Conditioned Entity Representation**

We aim to elicit dynamic entity representations through a query-conditioned attention mechanism. For each entity, we first calculate attention weights over all tokens in the user query  $Q$ , enabling adaptive focus on the query aspects most relevant to each entity. The resulted attention distribution is then used to compute a weighted sum of  $Q$ ’s token embeddings, which encodes the entity’s problem-specific semantic relevance. This design allows the representations to dynamically adjust based on different users’ needs. Specifically, the entity representation  $v_i$  is projected to the query vector via a multilayer perceptron module  $\mathcal{W}_q$ :  $q_i = \mathcal{W}_q(v_i)$ . The user query vectors  $Q$  are then projected into the key and value spaces as:  $k = \mathcal{W}_k(Q)$ ,  $v = \mathcal{W}_v(Q)$ . Attention weights  $\alpha_i$  between the  $i$ -th entity and each user query token are computed using scaled dot-product attention.

Then the conditioned entity vector  $z_i$  is weighted as:

$$z_i = \mathbf{v} \cdot \alpha_i = \mathbf{v} \cdot \text{softmax}\left(\frac{\mathbf{k}^\top \cdot \mathbf{q}_i}{\sqrt{d}}\right) \in \mathbb{R}^L \quad (1)$$

• **Step 3: Expectation-Guided Graph Activation**

Using the query-conditioned entity representation, this step aims to activate a expectation-guided knowledge graph that captures both the knowledge strength and relational strength among the Top-K entities. We argue this structure explicitly represents the user’s knowledge expectations regarding the query context.

We define the Expectation-Guided Graph Activation Matrix  $A = [a_{ij}]_{K \times K}$  over query-conditioned entity vectors  $\{z_i\}_{i=1}^K$ . For the main diagonal elements, i.e., the knowledge strength of each entity, the activation is computed by a multilayer perceptron as:

$$a_{ii} = \sigma(\mathcal{W}_{\text{diag}}(z_i)) \in (0, 1) \quad (2)$$

where  $\sigma$  denotes Sigmoid activation function. For off-diagonal elements, i.e., the relational strength of each knowledge pair, the activation is computed using a bilinear interaction:

$$a_{ij} = \sigma(\mathcal{W}_{\text{offdiag}}([z_i \parallel z_j])) \in (0, 1) \quad (3)$$

where  $[\cdot \parallel \cdot]$  denotes vector concatenation.

**3.2.3 Graph Prompt Tuning Task.** Our goal is to optimize the parameters in the Expectation-Guided Graph Encoder to predict an accurate Expectation-Guided Graph Activation Matrix. However, training such architecture presents a key challenge due to the unavailability of supervised labels for the activated graph. To this end, we propose a graph prompt tuning framework inspired by G-Retriever [9], where a graph neural network encodes the sub-graph into a continuous vector that serves as a learnable prompt for generation of a downstream language model. This design enables end-to-end training by propagating the language modeling loss backward through both the entity representation and graph activation components.

The graph encoding process is formally defined as:

$$h_g = \text{MeanPool}\left(\text{GNN}(\{z_i\}_{i=1}^K, A)\right) \in \mathbb{R}^{d_g} \quad (4)$$

where  $\{z_i\}_{i=1}^K$  are the query-conditioned entity vectors and  $A$  is the activation matrix,  $A$  is graph activation matrix, and  $\text{MeanPool}$  denotes mean pooling operation. The output dimension  $d_g$  matches the GNN’s hidden size.

To align with the downstream LLM’s embedding space, we conduct a multilayer perceptron on  $h_g : \hat{h}_g = \mathcal{W}_{\text{align}}(h_g)$ .

The final input  $x_{\text{final}}$  to the downstream LLM combines the soft prompt with the user query  $Q$  embeddings of the downstream LLM embedding layer.

The training objective minimizes the standard language modeling loss while freezing the downstream LLM’s parameters:

$$\mathcal{L}_{\text{tuning}} = - \sum_{t=1}^T \log p(y_t | y_{<t}, x_{\text{final}}) \quad (5)$$

To enhance the model training, we propose an Attention Consistency Loss to ensure distinct entities focus on different problem aspects by enforcing diversity in their attention distributions. For

each entity pair  $(i, j)$ , we penalize high cosine similarity between their attention vectors  $\alpha_i$  and  $\alpha_j$  if exceeding a threshold  $\tau$ :

$$\mathcal{L}_{\text{attn}_{ij}} = \max(0, \cos(\alpha_i, \alpha_j) - \tau) \quad (6)$$

$$\mathcal{L}_{\text{attn}} = \frac{1}{K(K-1)} \sum_{i \neq j} \mathcal{L}_{\text{attn}_{ij}}$$

The total loss combines this with the primary loss  $\mathcal{L}_{\text{tuning}}$ :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{tuning}} + \beta \mathcal{L}_{\text{attn}} \quad (7)$$

### 3.3 Reasoning Knowledge Graph Verification

Reasoning Knowledge Graph Verification evaluates and corrects model reasoning using the expectation scaffold constructed in Part 1. Given a chain-of-thought  $COT$  for query  $Q$ , we encode it into a reasoning activation matrix  $A_C$ , while the expectation encoder produces  $A_E$ . After aligning the two graphs into a shared entity space, we apply an adaptive fusion that learns confidence weights at both global and edge levels, yielding a fused graph  $A_F^*$ . This fused structure is converted into a soft graph prompt to guide answer generation, improving factual grounding, logical consistency, and robustness.

**3.3.1 Reasoning Chain Sampling.** We input the user query  $Q$  into a base Large Reasoning Model (LRM) and generate a chain of thought (CoT). The CoT is a short, readable trace of how the model moves from understanding the question to proposing an answer. To keep outputs focused, the prompt asks the model to first write the reasoning steps and then provide a brief final answer, separated by a dedicated delimiter. We also set a token budget to avoid overly long explanations, and for quality control, we perform a light filter: we drop empty or malformed traces and truncate overly long ones at sentence boundaries.

**3.3.2 User Query and Chain-of-thought Embedding.** We represent both the query  $Q$  and its CoT with a pretrained language model ( $LM$ ) in a shared space. Concretely, we take the last-layer hidden states as token-level vectors, giving  $Q = [q_1, \dots, q_{L_Q}]$  and  $COT = [c_1, \dots, c_{L_{COT}}]$ . This introduces no new parameters and keeps queries and rationales directly comparable.

**3.3.3 Adaptive Graph Fusion for Verification.** We first utilize the trained Expectation-Guided Graph Encoder to encode the user query  $Q$  to the expectation-guided graph structure  $A_E$ . We aim to use  $A_E$  to verify the model-generated reasoning chain  $C$  in a matrix fusion manner.

Leveraging the same architectural framework of Expectation-Guided Graph Encoder, we derive the graph activation matrix  $A_C$ . While  $A_C$  reflects the model’s actual reasoning trajectory,  $A_E$  represents user’s expectation for the given query. The adaptive fusion mechanism is designed to determine the confidence of the reasoning chain  $C$ ’s quality, allowing correction from  $A_E$  when  $C$  appears unreliable.

Specifically, given the reasoning chain’s embeddings  $C \in \mathbb{R}^{d \times L_c}$  and the problem representation  $Q_{[-1]} \in \mathbb{R}^d$ , we compute the query-conditioned reasoning representation  $z_r$  through another attention mechanism:

$$z_r = \text{softmax}\left(\frac{\mathcal{W}_q(Q_{[-1]})^T \mathcal{W}_k(C)}{\sqrt{d}}\right) \mathcal{W}_v(C) \quad (8)$$

where  $\mathcal{W}_q, \mathcal{W}_k, \mathcal{W}_v \in \mathbb{R}^{d \times d}$  are learnable query, key and value multilayer perceptron modules. Then we conduct sigmoid function to map the representation to the weight  $\alpha \in (0, 1)$ .

$$\alpha = \sigma(\mathcal{W}_{\text{map}}(z_r)) \quad (9)$$

The Top-K entities recovered from  $A_C$  and  $A_E$  may be inconsistent. To fuse  $A_C$  and  $A_E$  for verification, we first project them onto a shared space through zero-padding.

Let  $\mathcal{U} = \mathcal{V}^C \cup \mathcal{V}^E$  be the union of retrieved entities with  $|\mathcal{U}| = U \leq K_C + K_E$ . We remap  $A_C$  and  $A_E$  into the shared index space of  $\mathcal{U}$  by sparse placement (zero elsewhere), which yields  $\tilde{A}_C, \tilde{A}_E \in \mathbb{R}^{U \times U}$ :

$$\tilde{A}_S[i, j] = \begin{cases} A_S[m, n] & \text{if } (i, j) \text{ corresponds to } (V_m^S, V_n^S) \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

where  $S \in [C, E]$  and  $V^S$  denote entities in each subgraph.

Finally, we estimate edge-wise confidences:

$$\alpha_{ij} = \sigma(\mathcal{W}_{\text{map}}([z_r \parallel \phi_{ij}])) \quad (11)$$

with  $\phi_{ij}$  the pairwise features, and fuse via

$$A_F[i, j] = \alpha_{ij} \tilde{A}_C[i, j] + (1 - \alpha_{ij}) \tilde{A}_E[i, j] \quad (12)$$

followed by removing isolated nodes to obtain  $A_F^*$ .

**3.3.4 Response Generation.** After verification, our goal is to refine the final answer generation of the LRM. We again employ the graph prompt tuning strategy of section 3.2.3. We have two different points here: (1) the input to LRM comprises the soft graph vector, the user query  $Q$ 's embeddings, and the model-generated COT  $C$ 's embeddings; (2) we add the LoRA parameters to the LRM for better generation. Specifically, we apply LoRA to fine-tune the Large Reasoning Model (LRM) parameters, enabling it to learn task-specific answer patterns. We further adopt an asymmetric hierarchical learning rate strategy: a larger learning rate for LoRA parameters and a smaller learning rate for other custom modules. This design balances two needs: (1) Facilitating rapid adaptation of LRM's general representations to domain-specific terminology and logic via larger learning rates; (2) Ensuring stable convergence of randomly initialized custom modules through lower learning rates.

## 4 Experiments

We evaluate our model on real-world datasets to assess its performance in enhancing medical consultation. In particular, we aim to answer the following research questions: Q1: How effective is XGSV compared with state-of-the-art models? Q2: How do the main components of our model impact the performance? Q3: What is the impact of hyper-parameters?

### 4.1 Experimental Setup

**4.1.1 Datasets.** We evaluate our method in the knowledge-intensive medical field. Specifically, we use RJUA [17] and GenMedGPT-5K [31]. More details are shown in Appendix A.1.

- **RJUA** is a Chinese urological medical QA reasoning dataset with 1,705 training and 213 test samples, each containing a chain-of-thought derived from professional guidelines. We evenly split the training data for the expectation-guided graph encoder and the reasoning verification module. Since

no knowledge graph is provided, we extract 6,129 medical entities from the dataset using the Qwen-Plus API [33].

- **GenMedGPT-5K** consists of 5,452 ChatGPT-generated doctor-patient QA dialogues, split into 4,361 training and 1,091 test samples. The training set is evenly divided between the two modules, and EMCKG [31] is used as the external knowledge base, containing 5,802 triples, 6 relation types, and 1,123 entities.

**4.1.2 Baseline methods.** Test-time scaling methods include Self-Consistency [29] and Self-Refine [18]. **Fine-tuning methods** cover Prefix Tuning [15] and LoRA [10]. We also attempt to fine-tune the model using GRPO [23], an efficient RL method. We follow the training strategy of [2] to conduct RL training. **Knowledge Graph-based methods** include Think-on-Graph (ToG) [24] and G-Retriever variants [9]. Due to the lack of KG, we have not performed experiments on RJUA dataset. Additional baseline settings are provided in the appendix A.2.

**4.1.3 Evaluation Metrics.** We use BERTScore [36] and LLM-judged score [38] as evaluation metrics. BERTScore computes semantic similarity between generated and real answers using embeddings from pretrained BERT models. Three sub-metrics are reported in our paper, i.e., precision, recall and F1-Score. LLM-judged score evaluates model responses by prompting the advanced LLM to assign scores according to predefined evaluation criteria, enabling semantics-aware assessment. Herein, we adopt a 5-level scoring criterion to evaluate the quality of the model response and choose the Qwen-Plus [33] as the evaluation model. Additional information of evaluations are given in the appendix A.3.

**4.1.4 Implementation Details.** In expectation-guided graph generation part, we use Qwen3-Embedding-8B [37] as the pretrained embedding model, and Qwen3-8B (non-thinking mode) [32] as the downstream LLM. For the reasoning verification training, we use two open-source LRMs—Deepseek-R1-Distill-Qwen-7B and Qwen3-8B (thinking mode) [32] as base models. In reasoning verification, the learning rates for LoRA parameters and other parameters are respectively  $5e-4$  and  $5e-6$ . For all experiments, we set Top-K as 50 and  $\beta$  as 0.1. We run codes on a Linux server equipped with dual Intel Xeon E5-2650 CPUs (2.20 GHz) and NVIDIA A100 GPUs. Additional details are provided in the appendix.

### 4.2 Model Performance (Q1)

As observed from Tables 1 and 2, XGSV achieves top-tier performance across the RJUA and GenMedGPT-5K benchmarks on both Qwen3-8B and DeepSeek-R1-7B, ranking first or second across all major metrics. On Qwen3-8B, XGSV achieves a relative improvement of 5.77% in F1 over Prefix Tuning and a relative improvement of 0.244 LLM-Judges in Self Consistency on RJUA. Since RJUA lacks knowledge graphs, we do not report KG-based methods. On GenMedGPT-5K, it provides a 0.08% relative gain of F1 and 0.015 relative gain of LLM-Judge versus the strongest baselines. On DeepSeek-R1-7B, XGSV achieves an 8.32% relative F1 improvement on RJUA over the strongest baseline and ranks second on LLM-Judge—marginally below Self Consistency but substantially ahead of all others. On GenMedGPT-5K, it attains a 0.7% relative F1 gain over G-Retriever and ranks second in LLM-Judge performance, falling just slightly

**Table 1: Comparison of various methods on RJUA and GenMedGPT-5K benchmarks based on Qwen3-8B. The results cover different strategies. Metrics include BERTScore Precision (P), Recall (R), and F1, as well as LLM-Judge scores. Bold indicates the best results, while underlined denotes the second-best.**

| Type              | Model/Method                     | RJUA         |              |              |              | GenMedGPT-5K |              |              |              |
|-------------------|----------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                   |                                  | BertScore    |              |              | LLM-Judge    | BertScore    |              |              | LLM-Judge    |
|                   |                                  | P            | R            | F1           |              | P            | R            | F1           |              |
| Base              | Qwen [32]                        | 59.21        | 68.98        | 63.65        | 2.789        | 75.41        | 84.98        | 79.88        | 1.383        |
| Test time scaling | Self Consistency [29]            | 59.07        | 69.57        | 63.82        | <u>2.906</u> | 75.83        | 85.07        | 80.16        | 1.376        |
|                   | Self Refine [18]                 | 57.36        | 65.93        | 61.29        | 2.282        | 75.06        | 84.04        | 79.27        | 1.221        |
| Fine tuning       | Prefix Tuning [15]               | 70.55        | 71.47        | 70.79        | 2.268        | 86.22        | 87.62        | 86.88        | 1.369        |
|                   | LoRA [10]                        | 66.59        | 68.63        | 67.44        | 2.211        | 87.03        | 87.70        | 87.33        | 1.505        |
|                   | GRPO [23]                        | 61.64        | 67.79        | 64.41        | 1.953        | 83.74        | 87.14        | 85.36        | 1.481        |
| KG-based          | ToG [24]                         | -            | -            | -            | -            | 78.84        | 85.04        | 81.78        | 1.180        |
|                   | G-Retriever+PT [9]               | -            | -            | -            | -            | 75.52        | 84.89        | 79.88        | 1.223        |
|                   | G-Retriever+LoRA [9]             | -            | -            | -            | -            | <b>90.76</b> | <u>87.74</u> | <u>89.17</u> | <u>1.520</u> |
| <b>Ours</b>       | <b>XGSV (Qwen as base model)</b> | <b>76.97</b> | <b>76.43</b> | <b>76.56</b> | <b>3.150</b> | <u>90.32</u> | <b>88.33</b> | <b>89.25</b> | <b>1.535</b> |

**Table 2: Comparison of various methods on RJUA and GenMedGPT-5K benchmarks based on DeepSeek-R1-Distill-Qwen-7B. The results cover different strategies. Metrics include BERTScore Precision (P), Recall (R), and F1, as well as LLM-Judge scores. Bold indicates the best results, while underlined denotes the second-best.**

| Type              | Model/Method                         | RJUA         |              |              |              | GenMedGPT-5K |              |              |              |
|-------------------|--------------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|                   |                                      | BertScore    |              |              | LLM-Judge    | BertScore    |              |              | LLM-Judge    |
|                   |                                      | P            | R            | F1           |              | P            | R            | F1           |              |
| Base              | DeepSeek [4]                         | 61.78        | 69.45        | 65.27        | 1.948        | 78.64        | 84.75        | 81.54        | 1.301        |
| Test time scaling | Self Consistency [29]                | 59.99        | <u>70.73</u> | 64.84        | <b>2.826</b> | 76.34        | 85.03        | 80.42        | 1.349        |
|                   | Self Refine [18]                     | 60.38        | 68.89        | 64.30        | 1.812        | 76.76        | 84.02        | 80.20        | 1.218        |
| Fine tuning       | Prefix Tuning [15]                   | <u>65.97</u> | 69.19        | <u>67.35</u> | 1.742        | 86.18        | <b>87.56</b> | 86.83        | 1.272        |
|                   | LoRA [10]                            | 59.95        | 68.09        | 63.65        | 1.817        | 86.41        | 86.78        | 86.55        | <b>1.368</b> |
|                   | GRPO [23]                            | 62.68        | 65.41        | 63.87        | 1.808        | 83.05        | 86.46        | 84.68        | 1.346        |
| KG-based          | ToG [24]                             | -            | -            | -            | -            | 78.81        | 83.51        | 81.05        | 1.223        |
|                   | G-Retriever+PT [9]                   | -            | -            | -            | -            | 78.35        | 83.79        | 80.93        | 1.003        |
|                   | G-Retriever+LoRA [9]                 | -            | -            | -            | -            | <u>89.77</u> | <u>87.01</u> | <u>88.29</u> | 1.140        |
| <b>Ours</b>       | <b>XGSV (DeepSeek as base model)</b> | <b>76.85</b> | <b>74.87</b> | <b>75.67</b> | <u>2.671</u> | <b>90.59</b> | <u>87.54</u> | <b>88.99</b> | <u>1.353</u> |

behind LoRA but outperforming all other baselines. Taken together, the improvements are consistent across datasets, evaluation metrics, and backbone families, suggesting strong robustness and architectural generality, as well as the ability of XGSV to generalize beyond specific models or benchmarks and transfer effectively to a broad range of real-world, knowledge-intensive reasoning tasks.

### 4.3 Ablation Study (Q2)

To systematically evaluate the contribution of the expectation-guided self verification framework, we conduct ablation experiments by progressively removing key components: (i) w/o *EG*: disabling the expectation-guided knowledge graph in graph fusion (retaining only the reasoning graph  $A_C$ ), (ii) w/o *RG*: disabling the reasoning knowledge graph in graph fusion (retaining only the expected graph  $A_E$ ), and (iii) w/o *ALL*: removing fusion mechanism (base reasoning models). As summarized in Table 3, the removal of any single branch consistently degrades performance across metrics, underscoring the complementary benefits of both types of graph. Notably, removing the entire fusion leads to a pronounced performance drop, revealing that the base LRM alone is insufficient to perform well in domain reasoning tasks.

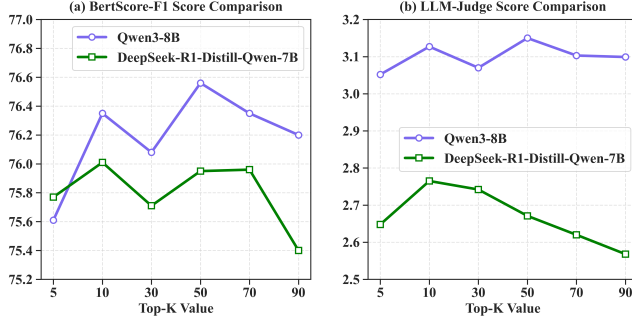
**Table 3: Ablation on different types of subgraphs on RJUA**

| Methods         | BertScore    |              |              | LLM-Judge    |
|-----------------|--------------|--------------|--------------|--------------|
|                 | P            | R            | F1           |              |
| XGSV (Qwen)     | <u>76.97</u> | <b>76.43</b> | <b>76.56</b> | <b>3.150</b> |
| w/o <i>EG</i>   | <b>77.26</b> | 75.44        | 76.19        | 3.127        |
| w/o <i>RG</i>   | 76.80        | 75.63        | 76.06        | 3.099        |
| w/o <i>ALL</i>  | 59.21        | 68.98        | 63.65        | 2.789        |
| XGSV (DeepSeek) | 76.85        | <b>74.87</b> | <b>75.67</b> | <b>2.671</b> |
| w/o <i>EG</i>   | <u>76.94</u> | 73.83        | 75.21        | 2.643        |
| w/o <i>RG</i>   | <b>78.33</b> | 73.09        | 75.48        | 2.610        |
| w/o <i>ALL</i>  | 61.78        | 69.45        | 65.27        | 1.948        |

Furthermore, we evaluate the importance of our adaptive fusion mechanism by replacing it with the baseline averaging operation:  $A_F = 0.5\hat{A}_C + 0.5\hat{A}_E$ . As reported in Table 4, the adaptive fusion module consistently surpasses this naive averaging on both BertScore-F1 and LLM-Judge, demonstrating that dynamically weighting contributions from  $\hat{A}_C$  and  $\hat{A}_E$  allows the model to emphasize task-relevant evidence while suppressing spurious edges.

**Table 4: Ablation study of graph fusion methods on RJUA**

| Model             | BertScore    |              |              | LLM-Judge    |
|-------------------|--------------|--------------|--------------|--------------|
|                   | P            | R            | F1           |              |
| XGSV (Qwen)       | <b>76.97</b> | <b>76.43</b> | <b>76.56</b> | <b>3.150</b> |
| w/ Average Fusion | 75.90        | 76.17        | 75.88        | 2.906        |
| XGSV (DeepSeek)   | <b>76.85</b> | <b>74.87</b> | <b>75.67</b> | <b>2.671</b> |
| w/ Average Fusion | <b>76.94</b> | 73.89        | 75.20        | 2.606        |

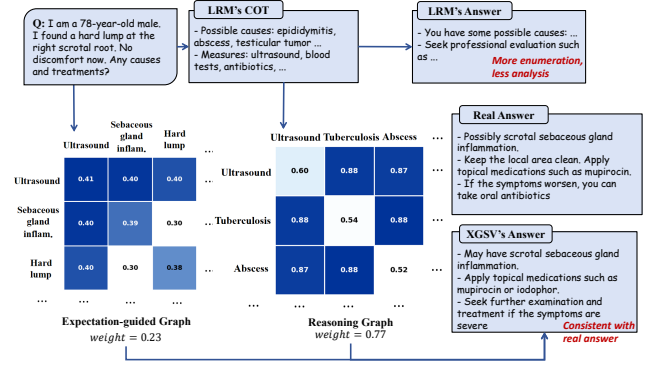
**Figure 3: Sensitivity Analysis on Top-K value on RJUA**

#### 4.4 Sensitivity Analysis (Q3)

We conducted an experiment on the RJUA dataset to systematically analyze how the number of retrieved entities (Top-K) impacts the efficacy of our retrieval-augmented generation. As shown in Figure 3, for both Qwen3-8B and DeepSeek-R1-Distill-Qwen-7B, performance on BertScore-F1 and LLM-Judge Score exhibits a clear, non-monotonic sensitivity curve: scores rise as  $K$  increases from a sparse setting, reach a stable peak, and then decline once the context becomes cluttered. This inverted-U pattern is robust across random seeds, demonstrating that while insufficient retrieval limits context, excessive retrieval introduces harmful distractions. Specifically, when the Top-K value is too small, the retrieved candidates under-cover relevant entities, severely limiting evidence aggregation and yielding suboptimal results due to information gaps. As  $K$  grows, coverage improves and the model benefits from richer, corroborating cues, thereby boosting performance and grounding the generation. However, pushing  $K$  beyond the optimal threshold admits redundant or tangential items, injecting noise that dilutes the attention mechanism and depresses precision. Therefore, tuning  $K$  is essential to effectively balance the trade-off between knowledge recall and noise. In our setting, moderate Top-K values (e.g., 10–50) deliver the best overall outcomes, effectively curbing latency and memory pressure while avoiding the pitfalls of both information scarcity and context saturation.

#### 4.5 Case Study

We present a case study on one sample from the RJUA dataset to illustrate our method’s efficacy. As shown in Figure 4, using Qwen3-8B as the base LRM, we analyzed a query of a 78-year-old male, where the ground truth identifies "scrotal sebaceous gland inflammation" as the most likely diagnosis. While the LRM generates diverse content suggesting various conditions like epididymitis or

**Figure 4: A case study of Qwen3-8B on RJUA**

tumors, it fails to focus on the target disease and lacks in-depth, differential reasoning. To rectify this, our XGSV method first converts the user query into an expectation-guided graph containing the core disease "scrotal sebaceous gland inflammation," essentially anchoring the diagnosis to the patient’s specific symptoms. It then transforms the LRM’s CoT into a reasoning graph, which is assigned a weight of 0.77 based on the original quality of the reasoning content. The strategic fusion of these two graphs constrains the inference process, guiding the final generation away from broad enumeration. This results in alignment with the ground truth and clearer, clinically actionable recommendations—such as specific topical treatments—demonstrating XGSV’s ability to correct reasoning drift and enhance domain-specific interpretability and reliability.

#### 5 Conclusion

We introduce a domain reasoning method, Expectation-Guided Self-Verification (XGSV), which explicitly encodes user expectations using an expectation-guided knowledge graph and aligns them with the LRM’s reasoning knowledge graph via adaptive, instance-wise graph fusion. Rather than relying solely on the model’s internal probabilities, this approach dynamically balances the user’s specific constraints against the model’s broad, generated logic to correct trajectory errors. This closed-loop self-verification progressively audits intermediate inferences against domain-specific priors, effectively suppressing spurious correlations, mitigating hallucinations, and improving long-horizon stability where standard chains of thought typically drift. Ablation analyses further confirm that each component (expectation graph, reasoning graph, and adaptive fusion) is indispensable for achieving this robust alignment, while efficiency profiling shows negligible overhead relative to naive models. More broadly, XGSV provides a general and interpretable paradigm for expectation-aware reasoning, with potential applicability to other high-stakes, knowledge-intensive domains beyond medical consultation.

#### Acknowledgments

This work was supported in part by the National Natural Science Foundation of China [Grants: 72522010, 72394371, and 72571071]. The computations in this research were performed using the CFFF platform of Fudan University.

## References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2024. Gpt-4 technical report. arXiv:2303.08774 [cs.CL] <https://arxiv.org/abs/2303.08774>
- [2] Junying Chen, Zhenyang Cai, Ke Ji, Xidong Wang, Wanlong Liu, Rongsheng Wang, Jianye Hou, and Benyou Wang. 2024. HuatuoGPT-o1, Towards Medical Complex Reasoning with LLMs. arXiv:2412.18925 [cs.CL] <https://arxiv.org/abs/2412.18925>
- [3] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Naveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. arXiv:2507.06261 [cs.CL] <https://arxiv.org/abs/2507.06261>
- [4] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 4171–4186.
- [6] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [7] Ethan Goh, Robert Gallo, Jason Hom, Eric Strong, Yingjie Weng, Hannah Kerman, Josephine A Cool, Zahir Kanjee, Andrew S Parsons, Neera Ahuja, et al. 2024. Large language model influence on diagnostic reasoning: a randomized clinical trial. *JAMA network open* 7, 10 (2024), e2440969–e2440969.
- [8] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv:2501.12948 [cs.AI]
- [9] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems* 37 (2024), 132876–132907.
- [10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [11] Xue Jiang, Yihong Dong, Lecheng Wang, Zheng Fang, Qiwei Shang, Ge Li, Zhi Jin, and Wenpin Jiao. 2024. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.
- [12] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. arXiv:2104.08691 [cs.CL] <https://arxiv.org/abs/2104.08691>
- [13] Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2025. Structured chain-of-thought prompting for code generation. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–23.
- [14] Junyi Li and Hwee Tou Ng. 2025. The Hallucination Dilemma: Factuality-Aware Reinforcement Learning for Large Reasoning Models. arXiv:2505.24630 [cs.CL] <https://arxiv.org/abs/2505.24630>
- [15] Xiang Lisa Li and Percy Liang. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. arXiv:2101.00190 [cs.CL] <https://arxiv.org/abs/2101.00190>
- [16] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. arXiv:1907.11692 [cs.CL] <https://arxiv.org/abs/1907.11692>
- [17] Shiwei Lyu, Chenfei Chi, Hongbo Cai, Lei Shi, Xiaoyan Yang, Lei Liu, Xiang Chen, Deng Zhao, Zhiqiang Zhang, Xiangguo Lyu, Ming Zhang, Fangzhou Li, Xiaowei Ma, Yue Shen, Jinjie Gu, Wei Xue, and Yiran Huang. 2023. RJUA-QA: A Comprehensive QA Dataset for Urology. arXiv:2312.09785 [cs.CL]
- [18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594.
- [19] Fnu Neha and Deepshikha Bhati. 2025. A Survey of DeepSeek Models. *Authorea Preprints* (2025).
- [20] Richard L Oliver. 1980. A cognitive model of the antecedents and consequences of satisfaction decisions. *Journal of marketing research* 17, 4 (1980), 460–469.
- [21] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems* 36 (2023), 53728–53741.
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347 [cs.LG] <https://arxiv.org/abs/1707.06347>
- [23] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL] <https://arxiv.org/abs/2402.03300>
- [24] Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. arXiv:2307.07697 [cs.CL] <https://arxiv.org/abs/2307.07697>
- [25] Zhongxiang Sun, Qipeng Wang, Haoyu Wang, Xiao Zhang, and Jun Xu. 2025. Detection and Mitigation of Hallucination in Large Reasoning Models: A Mechanistic Perspective. arXiv:2505.12886 [cs.AI] <https://arxiv.org/abs/2505.12886>
- [26] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. 2025. Kimi K2: Open Agentic Intelligence. arXiv:2507.20534 [cs.LG] <https://arxiv.org/abs/2507.20534>
- [27] Mickael Tordjman, Zelong Liu, Murat Yuce, Valentin Fauveau, Yunhao Mei, Jerome Hadjadji, Ian Bolger, Haidara Almansour, Carolyn Horst, Ashwin Singh Parihar, et al. 2025. Comparative benchmarking of the DeepSeek large language model on medical tasks and clinical reasoning. *Nature medicine* (2025), 1–1.
- [28] Alicia Y. Tsai, Adam Kraft, Long Jin, Chenwei Cai, Anahita Hosseini, Taibai Xu, Zemin Zhang, Lichan Hong, Ed H. Chi, and Xinyang Yi. 2024. Leveraging LLM Reasoning Enhances Personalized Recommender Systems. arXiv:2408.00802 [cs.IR] <https://arxiv.org/abs/2408.00802>
- [29] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=1PL1NIMMrw>
- [30] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. 2022. Finetuned Language Models Are Zero-Shot Learners. arXiv:2109.01652 [cs.CL] <https://arxiv.org/abs/2109.01652>
- [31] Yilin Wen, Zifeng Wang, and Jimeng Sun. 2024. MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Large Language Models. arXiv:2308.09729 [cs.AI] <https://arxiv.org/abs/2308.09729>
- [32] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [33] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024. Qwen2 technical report. arXiv:2407.10671 [cs.CL] <https://arxiv.org/abs/2407.10671>
- [34] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2025. Qwen2. 5 technical report. arXiv:2412.15115 [cs.CL] <https://arxiv.org/abs/2412.15115>
- [35] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [36] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations*.
- [37] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. arXiv:2506.05176 [cs.CL] <https://arxiv.org/abs/2506.05176>
- [38] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.

## A Experiment Details

### A.1 Data Preprocessing

**A.1.1 Reasons of Dataset Selection.** We aim to evaluate our method on the knowledge-intensive domain: medical consultation field. Ideally, the dataset is composed of two data sources: (1) query-cot-answer pairs  $\mathcal{D} = \{(Q_i, COT_i, A_i)\}_{i=1}^n$  and knowledge graphs; (2)  $\mathcal{G} = \{(h, r, t) \mid h, t \in \mathcal{E}, r \in \mathcal{R}\}$  with  $N$  entities and  $M$  relations.

However, collecting a single dataset that integrates both query-cot-answer pairs and knowledge graphs is challenging. To address this, we select two complementary open-source datasets: RJUA [17] and GenMedGPT-5K [31]. As illustrated in Table 5, the RJUA dataset lacks corresponding knowledge graph data but includes COT data, while GenMedGPT-5K provides knowledge graph data EMCKG but no COT data. This distinction leads to different settings of baseline methods for the two datasets. We will show the baseline settings in Section 3.2.

Table 5: Dataset Information

| Dataset      | Language | Num  | COT | KG  |
|--------------|----------|------|-----|-----|
| RJUA         | Chinese  | 1918 | Yes | No  |
| GenMedGPT-5K | English  | 5452 | No  | Yes |

**A.1.2 Entity Extraction on RJUA.** Since the RJUA dataset lacks a knowledge graph but our method depends on entities, we construct entities from the entire RJUA dataset using Qwen-PLUS [32], a high-capability LLM. Specifically, we apply the LLM to each query-cot-answer pair and prompt it to extract relevant entities following predefined rules.

Our prompt is as follows:

#### Entities Construction on RJUA Prompt

Please accurately extract the following 5 categories of medical entities from the medical text below and output them in JSON format:

# Entity Types:

Disease: e.g., "diabetes mellitus", "hypertension"

Symptom: e.g., "headache", "nausea"

Medicine: e.g., "amoxicillin", "insulin"

Diagnostic Test: e.g., "complete blood count", "magnetic resonance imaging"

Treatment: e.g., "surgery", "radiotherapy"

# Rules:

Only output entities explicitly mentioned in the text; do not infer entities not specifically mentioned. Use standard terminology (e.g., "gastroscopy" → "upper gastrointestinal endoscopy")

# Output Format:

"disease": [],

"symptom": [],

"medicine": [],

"diagnostic\_test": [],

"treatment": []

# Example:

Input: "User's question: I have heart pain, what should I do?; Doctor's reply: It is recommended to first do an electrocardiogram to rule out myocardial infarction, then take aspirin, and pay close attention to physical conditions"

Output:

"disease": ["myocardial infarction"],

"symptom": ["heart pain"],

"medicine": ["aspirin"],

"diagnostic\_test": ["electrocardiogram"],

"treatment": ["close observation"]

Our prompt is as follows:

#### Entity Description Prompt

You are a medical expert. Next, the user will input a medical concept. Please provide a detailed explanation of the concept within 100 words. Ensure the content is accurate and professional. Do not output other content. Directly output the explanation.

For example, if the user inputs 'diabetes', the ideal output is: 'Diabetes is a group of metabolic diseases characterized by chronic hyperglycemia, mainly caused by insufficient insulin secretion or impaired insulin action. Long-term hyperglycemia can cause multi-system damage, such as cardiovascular, renal, neurological and retinal lesions. The main types include type 1 (insulin-dependent) and type 2 (non-insulin-dependent), which require comprehensive management through diet control, drug therapy and blood glucose monitoring.'

If the user inputs a non-medical concept, please directly reply: 'This is a concept unrelated to medicine.'

### A.2 Baseline Methods

In this subsection, we present the baseline methods adopted in our experiments. These baselines are designed to span a broad spectrum of strategies, which can help the large reasoning models align with user expectations potentially — from test time scaling methods to fine-tuning methods — as well as knowledge-graph (KG) grounded techniques.

**A.2.1 Method Compatibility Across Different Datasets.** As noted in Section 3.1.1 "Reasons of Datasets Selection", our datasets cannot have both COT and KG simultaneously. Consequently, certain methods are incompatible with specific datasets.

For RJUA dataset, due to the lack of knowledge graph, KG-based methods can not be implemented. Other methods can be used normally.

For GenMedGPT-5K dataset, the test-time methods and KG-based methods can work properly. Fine-tuning methods for the LRM require COT data, but the GenMedGPT-5K dataset lacks such data—ordinarily rendering fine-tuning approaches inapplicable. To enrich our experimental scope, however, we implement specific format adjustments to enable fine-tuning methods to be applied to GenMedGPT-5K despite this limitation. The adjustments are detailed in Section 3.2.4 "Fine-tuning Methods".

**A.2.2 Base Models.** The base models used are:

- **DeepSeek-R1-Distill-Qwen-7B[4]:** A distilled variant of DeepSeek-R1 optimized for efficiency while maintaining strong reasoning ability under constrained compute budgets.
- **Qwen3-8B[32]:** A recent model in the Qwen3 series with improved multilingual and reasoning capability, chosen to verify the generality of methods across architectures.

We access base models via API, which ensures efficiency. To enhance the reliability and consistency of model outputs, the temperature is fixed at 0.2.

**A.2.3 Test-Time Scaling Methods.** Test-time scaling methods improve performance without modifying model weights. In our work, we use Self-Refine [18] and Self-Consistency [29].

- **Self-Refine:** this method iteratively improves the output by prompting the model to revise its own reasoning chain. In our work, two iterations were selected based on preliminary ablations, where three or more iterations yielded insufficient

refinement but significantly higher latency. In addition, the temperature is set to 0.2 to ensure the reliability and consistency of model outputs.

- **Self-Consistency:** samples multiple reasoning paths and selects the most consistent answer via majority voting. In our work, three samples balance diversity of reasoning paths and computational overhead. Sample temperature of 0.7 encourages variation in outputs while avoiding excessive randomness. Additionally, aggregate temperature of 0.2 ensures the reliability when aggregating multiple reasoning paths.

**A.2.4 Fine-tuning Methods.** Fine-tuning is employed to adapt the base models toward the target task by updating either a subset or all of the model parameters. In our work, we use parameter-efficient supervised fine tuning including LoRA [10] and prefix tuning [15] and reinforcement learning including GRPO [23].

- **Training Data Format:** Before stepping into specific methods, let's pay attention to data preprocessing for fine-tuning methods. For fine-tuning data preprocessing, examples with chain-of-thought (CoT), such as RJUA are formatted inline as `<think>cot</think>answer` to simulate the reasoning process and align with mainstream reasoning models' output style, while examples without CoT like GenMedGPT-5K still include the `<think>` tag, using the form `<think></think>answer`.
- **LoRA:** this method is employed to efficiently adapt large language models by injecting trainable low-rank matrices into specific attention projection layers, while keeping the majority of model parameters frozen. In our work, the learning rate of  $5 \times 10^{-5}$  is chosen to ensure stable convergence while avoiding overfitting. A batch size of 8 is selected due to GPU memory constraints during fine-tuning, with three epochs providing a balance between adaptation quality and training efficiency. The rank of 8 and  $\alpha = 16$  are standard settings that offer sufficient expressiveness for low-rank adaptation without introducing excessive parameters. A dropout rate of 0.05 is applied to improve generalization. The  $q\_proj$  and  $v\_proj$  modules are targeted as they control the query and value projections in attention layers, which are the most effective points for LoRA adaptation according to prior work.
- **Prefix-tuning:** this method is applied to adapt the model by optimizing a set of trainable prefix tokens that steer the model's generation process, while keeping the original model weights frozen. In our work, learning rate of  $2 \times 10^{-4}$  is chosen to accelerate convergence, as the number of trainable parameters is small. A batch size of 8 is set due to GPU memory constraints, while three epochs provide sufficient training iterations for the prefix parameters to stabilize. The number of virtual tokens is set to 20, which is large enough to capture task-relevant signals without consuming excessive context length. Prefix projection is enabled (True) to allow the virtual tokens to be projected into a space that better aligns with the model's internal representations, improving adaptation effectiveness.
- **GRPO (Group Relative Policy Optimization):** this is employed to align the model's outputs with a task-specific reward function by optimizing the policy parameters through

reinforcement learning. This method compares multiple generated responses for the same query, assigns rewards based on a scoring function, and updates the policy to improve the likelihood of producing higher-reward outputs. In our work, the number of generations is set to 4 to provide sufficient diversity for relative reward comparison without incurring excessive computation. Two epochs are used to avoid overfitting while ensuring adequate policy updates. The learning rate of  $5 \times 10^{-6}$  is chosen for stable convergence in reinforcement learning. A batch size of 4 balances GPU memory usage and training efficiency. The temperature is set to 0.5 to encourage response diversity during generation, which is essential for effective reward-based optimization.

Additionally, following the strategy used in HuaTuo-o1 [2], we adopt a two-stage tuning approach: we use half of our training data for LoRA and the other half for GRPO. We also employ a five-point scoring system—where rewards are generated by the Qwen-Plus model. The corresponding prompt is shown below.

#### GRPO Reward Prompt

```
### Task Definition: Medical Q&A Answer Consistency Scoring
System (based on reference answer)
You are a medical Q&A quality evaluation expert. You need to
judge the model-generated answer (generated_answer) against the
authoritative reference answer (reference_answer).
### 1. Scoring Guidelines
Please strictly use the reference_answer as the core standard. The
specific rules:
- 5 points: Completely aligned. ...
- 4 points: Highly aligned. ...
- 3 points: Mostly aligned. ...
- 2 points: Low alignment. ...
- 1 point: Completely misaligned. ...
### 2. Evaluation Criteria (strictly based on reference_answer)
- Does it cover the core conclusion completely?
- Does it include key information?
~ ...
### 3. Output format
Output the score directly, e.g.: 5
```

**A.2.5 KG-based Methods.** Knowledge graph (KG)-based methods enhance model reasoning by integrating structured, relational information from external knowledge graphs. In our work, we use Think on Graph (ToG) [24] and G-retriever [9].

- **Think on Graph (ToG):** ToG performs reasoning by constructing a tree of subgraphs over a knowledge graph (KG). At each step, the method expands nodes along promising reasoning paths guided by beam search and incorporates retrieved KG triples into the reasoning process. This allows the model to execute structured, multi-hop reasoning grounded in factual relations. In our work, beam size of 3 provides sufficient exploration of alternative reasoning paths while avoiding combinatorial explosion. The maximum depth is set to 3 to capture multi-hop reasoning without introducing excessive graph expansion that could dilute relevant information. An exploration temperature of 0.4 encourages diversity in candidate expansions, while the reasoning temperature of 0.0 enforces deterministic reasoning once candidate paths are selected, improving answer stability.

- **G-Retriever:** G-Retriever is a GraphRAG method. It retrieves subgraphs from knowledge graphs via the PCST algorithm based on user queries, converts them to graph prompts using graph neural networks, and integrates them with large models for prompt learning—enhancing RAG’s knowledge graph utilization. We implemented it on the GenMedGPT-5K dataset with parameters: Top-K (entity) = 10, Top-E (edge) = 6, edge cost = 0.5. Both LoRA and pure prompt tuning are supported, with a shared learning rate of 5e-5. LoRA configurations: rank = 8,  $\alpha$  = 16, dropout = 0.05, targeting  $q\_proj$  and  $v\_proj$  modules.

### A.3 Evaluation

Our evaluation of each method’s performance focuses on the quality of generated answers relative to reference answers. Among the baseline methods, some produce both chain-of-thought reasoning processes and final answers. For evaluation purposes, we solely consider the final answers. The metrics used in our assessment include BERTScore [36] and LLM-Judge score [38].

**A.3.1 BERTScore.** BERTScore leverages embeddings from pre-trained BERT models to calculate the semantic similarity between generated responses and reference answers. For the Chinese RJUA dataset, we adopt the bert-base-chinese model [5]; while for the GenMedGPT-5K dataset, the RoBERTa-large model [16] is used for computation.

For each pair of generated and reference answers, it outputs three key metrics: Precision, Recall, and F1-Score. The final evaluation result is determined by averaging these three metrics across all samples in the dataset.

**A.3.2 LLM-Judge Score.** The LLM-judged score assesses model responses by prompting an advanced large language model (LLM) to assign scores based on predefined evaluation criteria, thereby enabling semantics-aware assessment. In this study, we adopt a 5-level scoring criterion to evaluate the quality of model responses and select Qwen-Plus [33] as the evaluation model. For each sample, a quality score is generated, and the average score across all samples is taken as the overall metric for the method on the dataset.

Our prompt for LLM Judge is as follows:

#### LLM-Judge Prompt

```
### Task Definition: Medical Q&A Answer Consistency Scoring System
(with reference answer as core benchmark)
You are a medical Q&A quality assessment expert. You will give an integer score between 1-5 (1 being the lowest, 5 being the highest) from the perspectives of content focus, core information coverage, and expression consistency.

### I. Scoring Instructions
Please strictly use the core content, information scope, and expression focus of the reference answer as criteria to judge the consistency of the generated answer. The specific standards are as follows:
- 5 points: Completely consistent. ...
- 4 points: Highly consistent. ...
- 3 points: Basically consistent. ...
- 2 points: Lowly consistent. ...
- 1 point: Completely inconsistent. ...

### II. Output Format
Please output strictly in the following format: Directly output the score, e.g., 5
```

## B Additional Experimental Results

### B.1 Generalization: Results on Financial and Legal Reasoning Datasets

Table 6 reports additional generalization results on financial (Finance-R1-Distill-Data) and legal (LawGPT) reasoning datasets. Compared with the base model and existing effective baselines, our method achieves consistently strong performance across both domains. Notably, while some baselines show competitive results on individual metrics, they tend to exhibit unstable behavior across different datasets, whereas our approach demonstrates more robust cross-domain generalization.

**Table 6: Additional Results on Financial and Legal Reasoning Datasets (DeepSeek-R1-Distill-Qwen-7B). Best results are in bold and second-best results are underlined.**

| Method     | Finance_R1-Distill_Data |              | LawGPT       |              |
|------------|-------------------------|--------------|--------------|--------------|
|            | F1 (%)                  | LLM-Judge    | F1 (%)       | LLM-Judge    |
| Base model | 86.51                   | 3.117        | 56.47        | 1.895        |
| SR         | 85.46                   | 2.512        | 50.46        | 1.589        |
| SC         | 86.21                   | <b>4.111</b> | 60.08        | <u>2.426</u> |
| LoRA       | 86.19                   | 2.665        | 72.58        | 1.920        |
| Prefix     | <u>87.35</u>            | 3.209        | <u>77.02</u> | 2.068        |
| Ours       | <b>88.03</b>            | <u>3.224</u> | <b>92.92</b> | <b>2.752</b> |

### B.2 Efficiency: Inference Latency Analysis

To evaluate the efficiency of our method, we report the inference latency on 100 samples of the legal reasoning dataset (LawGPT). The expectation graph encoder and the reasoning graph encoder can be executed in parallel, resulting in only a moderate increase in inference time compared to the base models.

**Table 7: Inference Latency Comparison on the Law Dataset**

| Model                       | Base   | Ours(XGSV) | Overhead |
|-----------------------------|--------|------------|----------|
| DeepSeek-R1-Distill-Qwen-7B | 129.2s | 150.0s     | +16.10%  |